Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 5

Logistic Regression

Background: Generative and Discriminative Classifiers

Logistic Regression

- Important analytic tool in natural and social sciences
- Baseline supervised machine learning tool for classification
- Is also the foundation of neural networks
- LR can be used to classify an observation into one of two classes or into one of many classes (multinomial LR).

Generative and Discriminative Classifiers

- Naïve Bayes is a **generative** classifier
- by contrast:
- Logistic regression is a **discriminative** classifier

Generative and Discriminative Classifiers

Suppose we're distinguishing cat from dog images



imagenet



imagenet

Generative Classifier

- Build a model of what's in a cat image
  - Knows about whiskers, ears, eyes
  - Assigns a probability to any image:
    - how cat-y is this image?
- Also build a model for dog images





Now given a new image:

Run both models and see which one fits better

Discriminative Classifier

Just try to distinguish dogs from cats





Oh look, all dog images (happen to) have collars!
Let's ignore everything else

Finding the correct class c from a document d in Generative vs Discriminative Classifiers

- Naive Bayes

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

- Logistic Regression

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(c|d)}^{\text{posterior}}$$

Components of a probabilistic machine learning classifier

Given m input/output pairs $(x^{(i)}, y^{(i)})$:

1. A **feature representation** of the input.

   For each input observation $x(i)$, a vector of features $[x_1, x_2, \ldots, x_n]$.
   Feature i for input $x^{(j)}$ is x_i, more completely $x_i^{(j)}$, or sometimes $f_i(x)$.

2. A **classification function** that computes $\hat{y}$, the estimated class, via $p(y|x)$, like the **sigmoid** or **softmax** functions.

3. An objective function for learning, like **cross-entropy loss**.

4. An algorithm for optimizing the objective function: **stochastic gradient descent**.

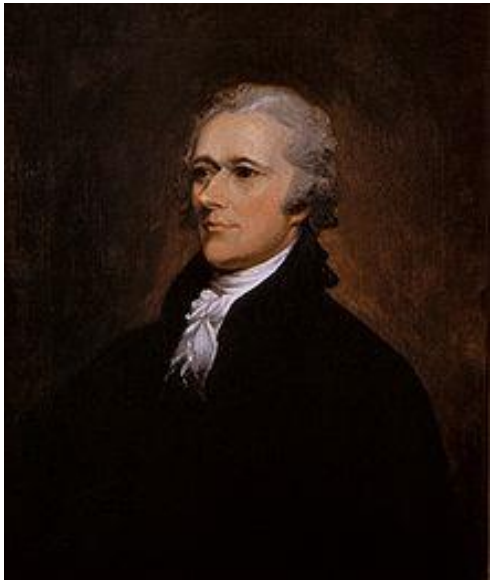
The two phases of Logistic Regression

- **Training**: we learn weights *w* and *b* using **stochastic gradient descent** and **cross-entropy loss**.
- **Test**: Given a test example *x,* we compute $p(y|x)$ using learned weights *w* and *b*, and return whichever label ($y = 1$ or $y = 0$) has higher probability.

Classification in Logistic Regression

Classification Reminder

- Positive/negative sentiment
- Spam/not spam
- Authorship attribution (Hamilton or Madison?)



Alexander Hamilton

Text Classification: definition

- *Input*:
  - a document x
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
- *Output*: a predicted class $\hat{y} \in C$

Binary Classification in Logistic Regression

- Goal of binary LR: train a classifier that can make a binary decision about the class of a new input observation
- Given a series of input/output pairs: $(x^{(i)}, y^{(i)})$
- For each observation $x^{(i)}$
  - We represent $x^{(i)}$ by a **feature vector** $[x_1, x_2, \ldots, x_n]$
  - We compute an output: a predicted class $\widehat{y^{(i)}} \in \{0,1\}$

Binary Classification in Logistic Regression

- We want to know the probability P(y = 1|x) that this observation is a member of the class.
  - E.g. "positive sentiment" versus "negative sentiment";
  - the features represent counts of words in a document;
  - P(y = 1|x): probability that the document has positive sentiment;
  - P(y = 0|x): probability that the document has negative sentiment.

Features in logistic regression

- For feature x_i, weight w_i tells us how important x_i is.
  - x_i ="review contains '`awesome`": $w_i = +10$
  - x_j ="review contains '`abysmal`": $w_j = -10$
  - x_k ="review contains '`mediocre`": $w_k = -2$

Logistic Regression for one observation

- Input observation: vector $x = [x_1, x_2, \ldots, x_n]$
- Weights: one per feature: $W = [w_1, w_2, \ldots, w_n]$
  - Sometimes the weights are called $\theta = [\theta_1, \theta_2, \ldots, \theta_n]$
- Output: a predicted class $\hat{y} \in \{0,1\}$

(For multinomial logistic regression: $y \in \{0, 1, 2, 3, 4\}$)

- The output is considered as a linear function of the output

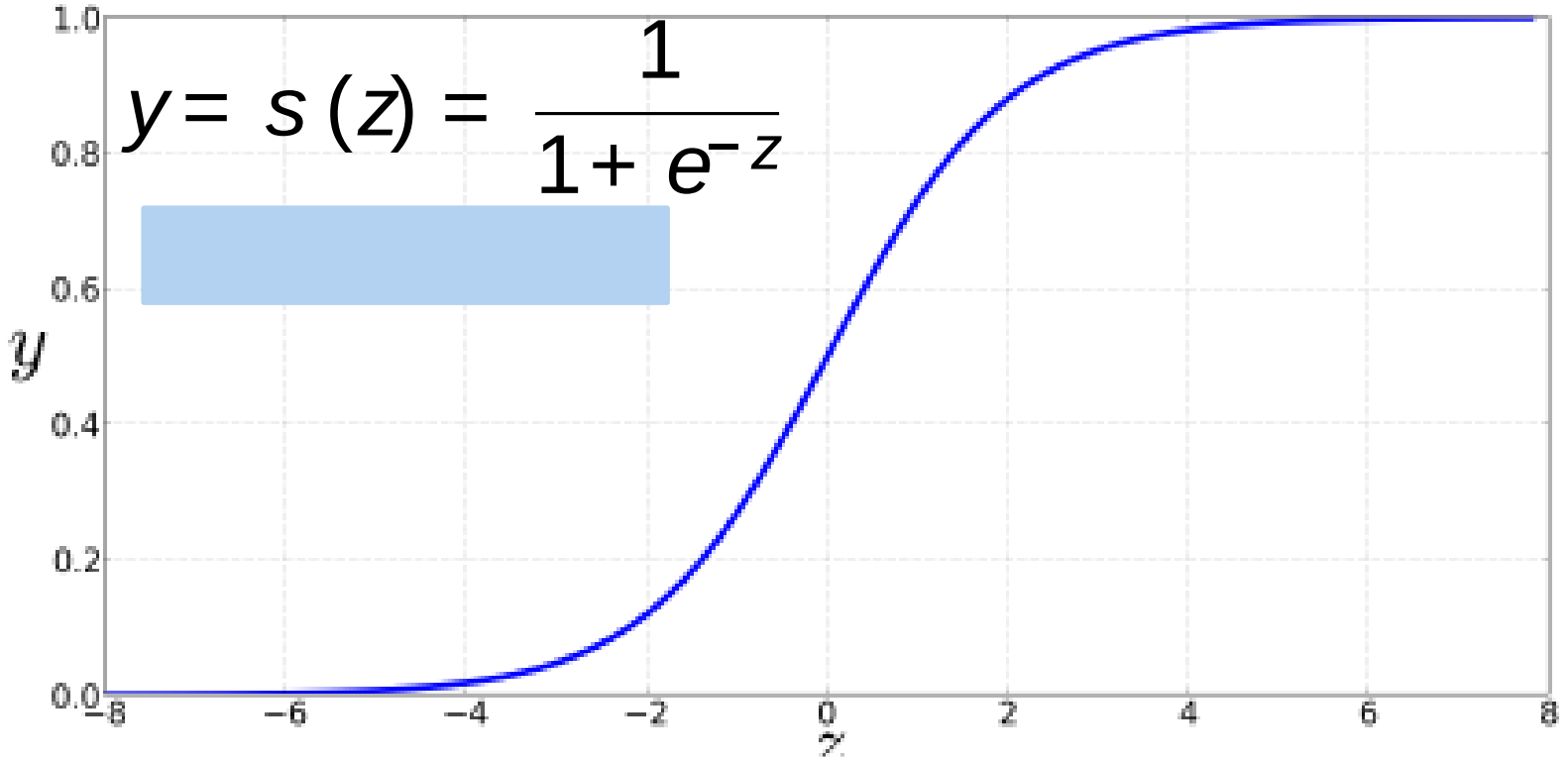
Problem

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b \qquad \text{so} \qquad z = \boldsymbol{w}.\boldsymbol{x} + b$$

- Inclusion of the *weights* and the *bias. (Note the dot product on the RHS)*
- But, z isn't a probability, it's just a number!
- We need to decide: if this sum is high, we say y=1; if low, then y=0
- Solution: use a function of z that goes from 0 to 1

$$y = s(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+\exp(-z)}$$

The very useful sigmoid or logistic function



Turning a probability into a classifier

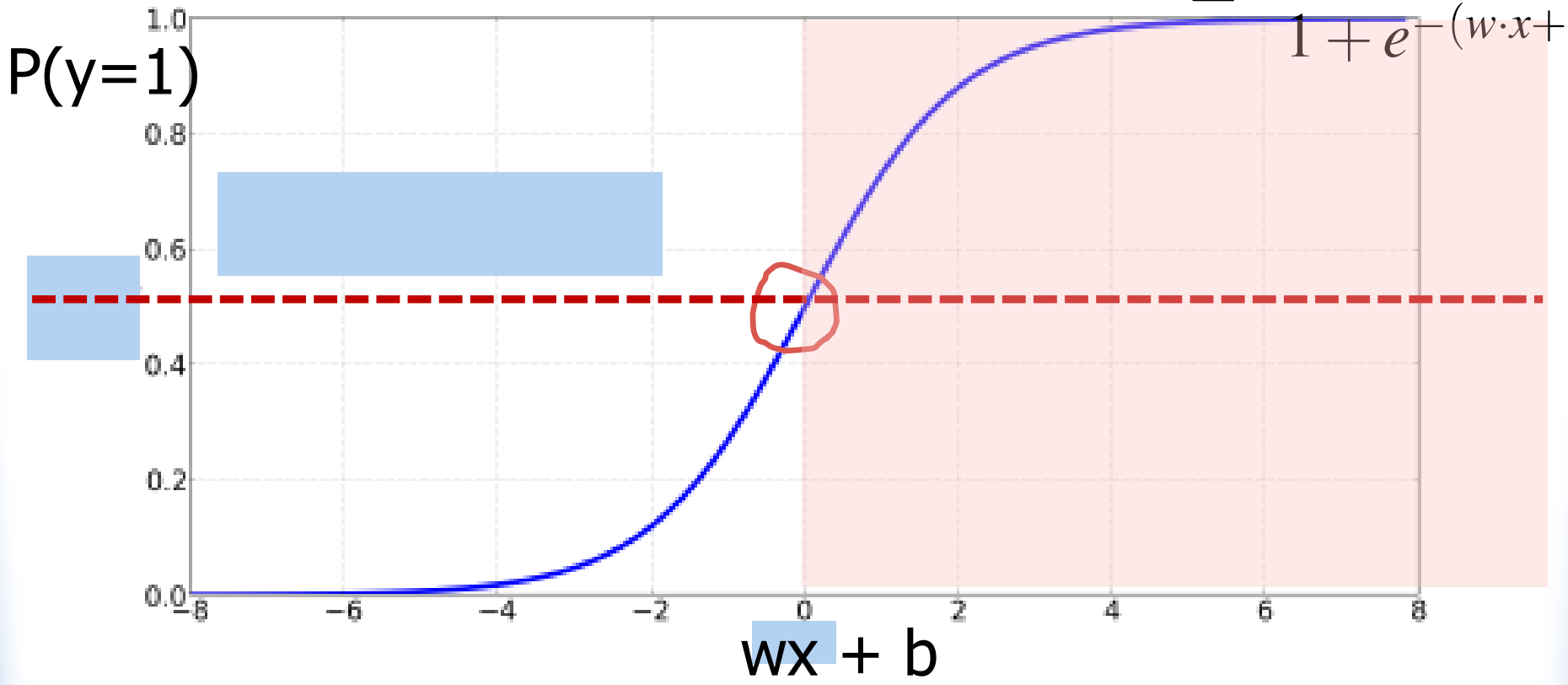
$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

0.5 here is called the **decision boundary**

The probabilistic classifier

$$P(y=1) = \sigma(w \cdot x + b)$$

$$= \frac{1}{1+e^{-(w \cdot x + b)}}$$



Turning a probability into a classifier

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

if $w \cdot x + b > 0$

if $w \cdot x + b \leq 0$

Logistic Regression: A Sentiment Classification Example

Sentiment Example

- Consider a binary sentiment classification on movie review texts
- We would like to know whether to assign the sentiment class + or - to a review document *doc*.

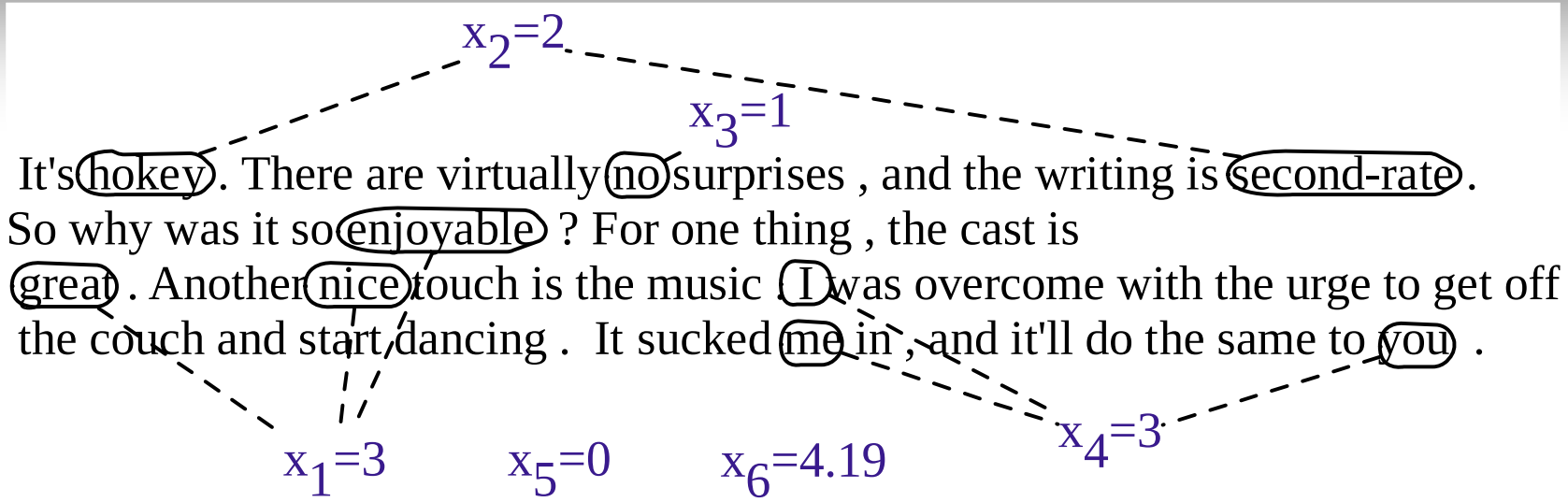
Fig. 5.2

- **It's hokey . There are virtually no surprises , and the writing is second-rate . So why was it so enjoyable ? For one thing , the cast is great . Another nice touch is the music . I was overcome with the urge to get off the couch and start dancing . It sucked me in , and it'll do the same to you .**

Sentiment Example

- We decided to represent each input observation (i.e. review) by the 6 features x_1 ... x_6 of the input shown in the following table.

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)



Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(66) = 4.19$

Classifying sentiment for input x

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(66) = 4.19$

Suppose we have learned $w = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$ and $b = 0.1$

Classifying sentiment for input x

$$p(+|x) = P(Y=1|x) = s(w \cdot x + b)$$
$$= s([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1)$$
$$= s(.833)$$
$$= 0.70$$

$$p(-|x) = P(Y=0|x) = 1 - s(w \cdot x + b)$$
$$= 0.30$$

Results rounded here to 2 decimal places.

Building features for LR for any classification task: e.g. period disambiguation

End of sentence

This ends in a period.

The house at 465 Main St. is new.

Not end of sentence

$$x_1 = \begin{cases} 1 \; if \; "Case(w_i) = Lower" \\ 0 \qquad\qquad otherwise \end{cases}$$

perhaps with a +ve weight

$$x_2 = \begin{cases} 1 \; if \; "w_i \in AcronymDict" \\ 0 \qquad\qquad otherwise \end{cases}$$

perhaps with a -ve weight

$$x_3 = \begin{cases} 1 \; if \; "w_i = St. \; and \; Case(w_{i-1}) = Cap" \\ 0 \qquad\qquad otherwise \end{cases}$$

Learning in Logistic Regression

How to get the W's?

- Supervised classification: we know the correct label y (either 0 or 1) for each x.
- What the system produces is an estimate, $\hat{y}$
- We want to set w and b to minimize the **distance** between our estimate $\hat{y}^{(i)}$ and the true $y^{(i)}$.
- We need a distance estimator: a **loss function** or a **cost function**
- We need an optimization algorithm to update w and b to minimize the loss.

Learning Components

- A loss function:
  - **cross-entropy loss**
- An optimization algorithm:
  - **stochastic gradient descent**

Cross-Entropy Loss

The distance between $\hat{y}$ and y

- We want to know how far is the classifier output (the predicted probability of the positive class, i.e. y=1):

  $\hat{y} = \sigma(w \cdot x + b)$

  from the true output (the actual label, 0 or 1):

  y

- We'll call this difference:

  $L(\hat{y}, y)$ = how much $\hat{y}$ differs from the true *y*

- We choose the parameters ***w***, ***b*** that maximize the log probability of the true *y* labels in the training data given the observations *x*

Deriving cross-entropy loss for a single observation x

Goal: Maximize the probability of the correct label $p(y|x)$ in the case of a binary classification

Maximize: $$p(y|x) = \hat{y}^y (1-\hat{y})^{1-y}$$

- Now take the log of both sides (mathematically handy)

So maximize: $$\log p(y|x) = \log \left[\hat{y}^y (1-\hat{y})^{1-y}\right]$$
$$= y \log \hat{y} + (1-y) \log(1-\hat{y})$$

- Whatever values maximize log p(y|x) will also maximize p(y|x)
- if y=1, this simplifies to $\hat{y}$; if y=0, this simplifies to 1– $\hat{y}$

Deriving cross-entropy loss for a single observation x

Maximize: $\log p(y|x) = \log \left[\hat{y}^y (1-\hat{y})^{1-y}\right]$

$= y \log \hat{y} + (1-y) \log(1-\hat{y})$

- Now flip sign to turn this into a loss function: something to minimize
- **Cross-entropy** Minimize:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

- Or, plugging in the definition of $\hat{y}$:

$$L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log(1-\sigma(w \cdot x + b))]$$

Application to the sentiment example

- We want the loss to be:
  - smaller if the model estimate is close to correct
  - bigger if the model is confused
- Let us first suppose the true label of this is y=1 (positive)

```
It's hokey . There are virtually no surprises
, and the writing is second-rate . So why was
it so enjoyable ? For one thing , the cast is
great . Another nice touch is the music . I
was overcome with the urge to get off the
couch and start dancing . It sucked me in ,
and it'll do the same to you .
```

Application to the sentiment example?

- True value is y=1. How well is our model doing?

$$
\begin{aligned}
p(+|x) = P(Y=1|x) &= \sigma(w \cdot x + b) \\
&= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\
&= \sigma(.833) \\
&= 0.70
\end{aligned}
$$

- So the loss is:

$$
\begin{aligned}
L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))] \\
&= -[\log \sigma(w \cdot x + b)] \\
&= -\log(.70) \\
&= .36
\end{aligned}
$$

Application to the sentiment example?

- Suppose the true value instead was y = 0.

$$p(-|x) = P(Y=0|x) = 1 - s(w \cdot x + b)$$
$$= 0.30$$

- So the loss is:

$$L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))]$$
$$= -[\log (1 - \sigma(w \cdot x + b))]$$
$$= -\log(.30)$$
$$= 1.2$$

Application to the sentiment example?

- The loss when the model was right (if true y=1)

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log \sigma(w \cdot x + b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

- Is lower than the loss when model was wrong (if true y=0):

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log (1 - \sigma(w \cdot x + b))] \\ &= -\log (.30) \\ &= 1.2 \end{aligned}$$

- Sure enough, the loss was bigger when the model was wrong!

Stochastic Gradient Descent

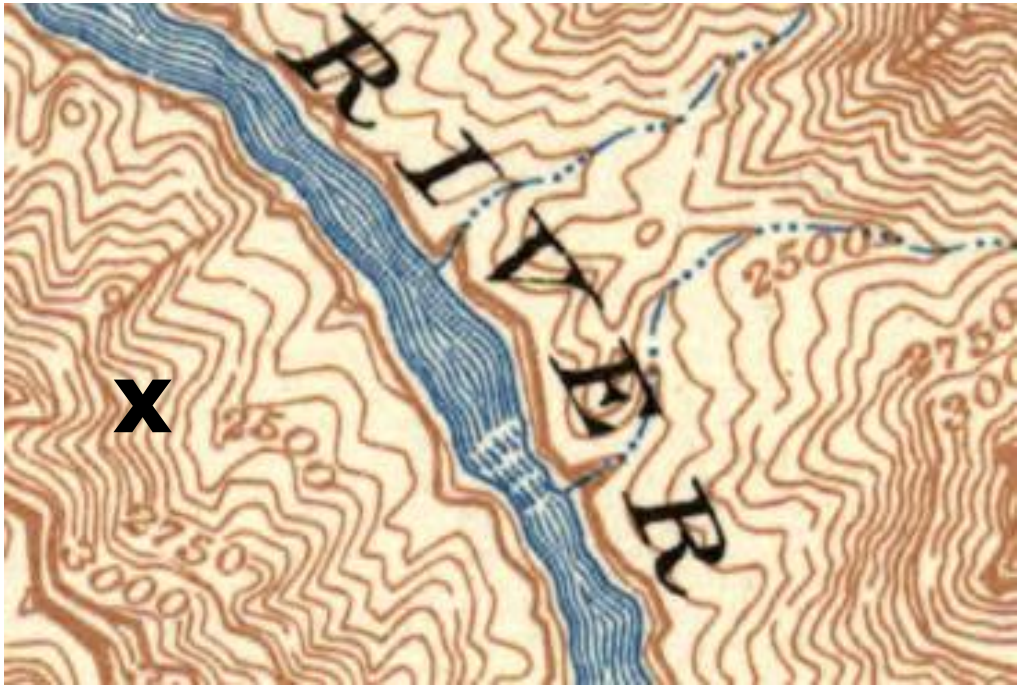
Our goal: minimize the loss

- Let's make explicit that the loss function in parameterized by weights θ=(w,b)
- We want the weights that minimize the loss, averaged over all examples:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^{m} L_{\text{CE}}\left(f\left(x^{(i)};\theta\right), y^{(i)}\right)$$

Intuition of gradient descent

- How do I get to the bottom of this



Look around me 360°

Find the direction of steepest slope

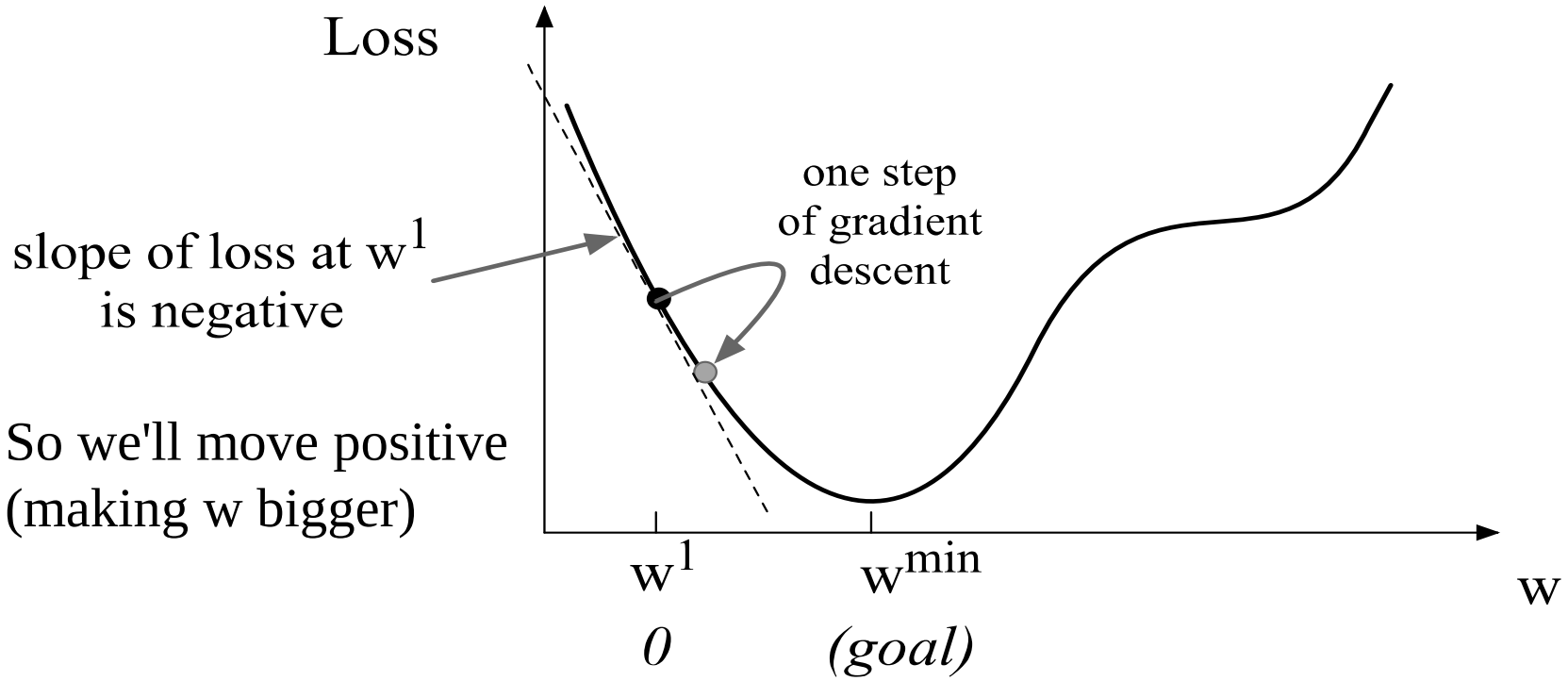
Go that way

Our goal: minimize the loss

- For logistic regression, the loss function is **convex**
- A convex function has just one minimum
- Gradient descent starting from any point is guaranteed to find the minimum
  - (The Loss for multi-layer neural networks is non-convex)

Visualization for a single scalar w

Q: Given current w, should we make it bigger or smaller?
A: Move *w* in the reverse direction from the slope of the function



where w^1 is the initial value of w

Gradients

- The **gradient** of a function of many variables is a vector pointing in the direction of the greatest increase in a function.
- **Gradient Descent**: Find the gradient of the loss function at the current point and move in the **opposite** direction.

Amount of move in that direction ?

- The value of the gradient (slope in our example) $\frac{d}{dw} f(x; w)$ weighted by a **learning rate** η
- Higher learning rate means move *w* faster

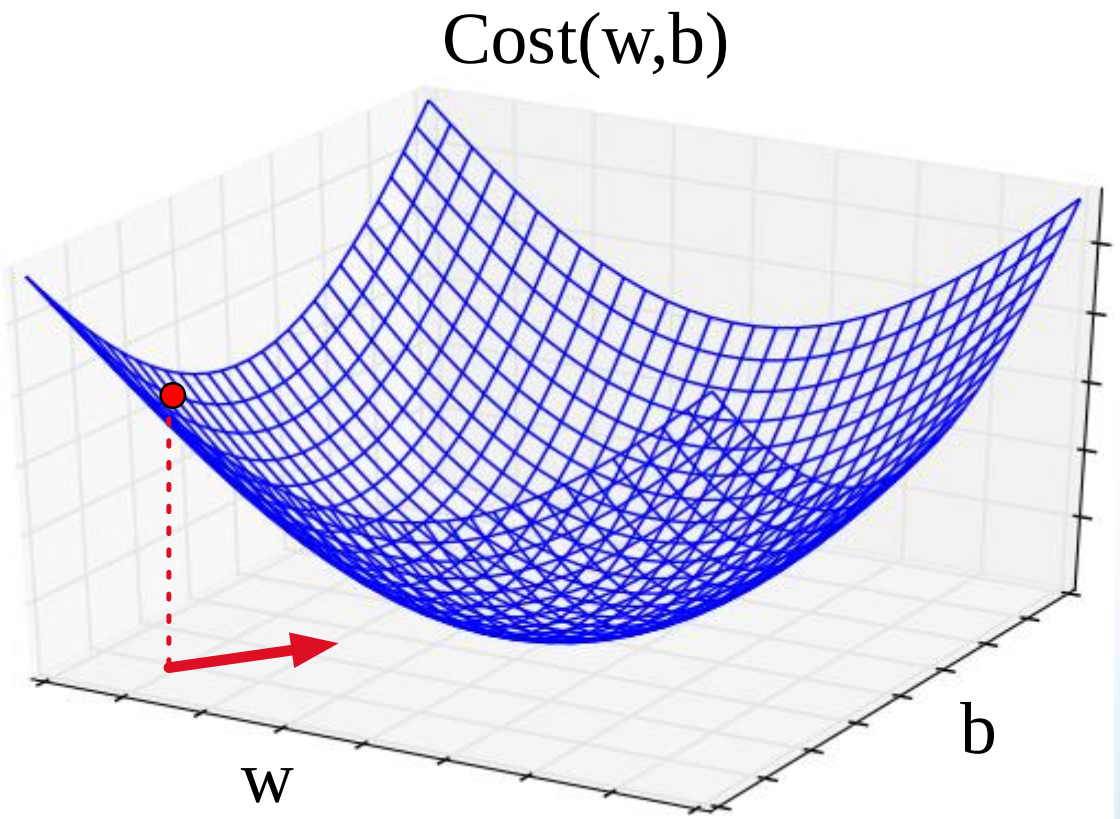
$$w^{t+1} = w^t - h \frac{d}{dw} f(x; w)$$

Case of N dimensions

- We want to know where in the N-dimensional space (of the N parameters that make up θ) we should move.
- The gradient is just such a vector; it expresses the directional components of the sharpest slope along each of the N dimensions.

Visualizing 2 dimensions, w and b

- Visualizing the gradient vector at the red point
- It has two dimensions shown in the x-y plane



Real gradients

- Are much longer; lots and lots of weights
- For each dimension w_i the gradient component i tells us the slope with respect to that variable.
  - "How much would a small change in w_i influence the total loss function L?"
  - We express the slope as a partial derivative of the loss with respect to w_i
- The gradient is then defined as a vector of these partial derivatives.

The gradient

We'll represent $\hat{y}$ as $f(x; \theta)$ to make the dependence on θ more obvious:

$$\nabla L(f(x;\theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x;\theta), y) \\ \frac{\partial}{\partial w_2} L(f(x;\theta), y) \\ \cdot \\ \cdot \\ \cdot \\ \frac{\partial}{\partial w_n} L(f(x;\theta), y) \\ \frac{\partial}{\partial b} L(f(x;\theta), y) \end{bmatrix}$$

The final equation for updating θ based on the gradient is thus

$$\theta^{t+1} = \theta^t - \eta \nabla \mathrm{L}(\mathrm{f}(\mathrm{x};\ \theta), y)$$

Partial derivatives for logistic regression

The loss function

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

The elegant derivative of this function:

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Algorithm for Stochastic Gradient Descent

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x, y) **returns** θ

where: L is the loss function
f is a function parameterized by θ
x is the set of training inputs $x^{(1)}$, $x^{(2)}$,..., $x^{(m)}$
y is the set of training outputs (labels) $y^{(1)}$, $y^{(2)}$,..., $y^{(m)}$

$\theta \leftarrow 0$

repeat til done # see caption

For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)

1. Optional (for reporting): # How are we doing on this tuple?
   Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?
   Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$) from the true output $y^{(i)}$?
2. $g \leftarrow \nabla_\theta L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss?
3. $\theta \leftarrow \theta - \eta\, g$ # Go the other way instead

return θ

Hyperparameters

- The learning rate η is a **hyperparameter**
  - too high: the learner will take big steps and overshoot
  - too low: the learner will take too long
- More on hyperparameters in Chapter 7
- Instead of being learned by algorithm from supervision (like regular parameters), they are chosen by the algorithm designer (or grid search).

Stochastic Gradient Descent: An example and more details

Working through an example

- One step of gradient descent
- A mini-sentiment example, where the true y=1 (positive)
- Two features:

$x_1 = 3$ (count of positive lexicon words)

$x_2 = 2$ (count of negative lexicon words)

Assume 3 parameters (2 weights and 1 bias) in Θ^0 are set to zero: $w_1 = w_2 = b = 0$

And learning rate $\eta = 0.1$

Example of gradient descent

- Step to update θ:

$w_1 = w_2 = b = 0$;
$x_1 = 3$; $x_2 = 2$

$$\theta^{t+1} = \theta^t - \eta \nabla \mathrm{L}(\mathrm{f}(x^{(i)}; \theta), y^{(i)})$$

where $$\frac{\partial L_{\mathrm{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

- Gradient vector has 3 dimensions:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\mathrm{CE}}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{\mathrm{CE}}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{\mathrm{CE}}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y) x_1 \\ (\sigma(w \cdot x + b) - y) x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1) x_1 \\ (\sigma(0) - 1) x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5 x_1 \\ -0.5 x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Example of gradient descent

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

- Now that we have a gradient, we compute the new parameter vector θ^1 by moving θ^0 in the opposite direction from the gradient:

$$\theta^{t+1} = \theta^t - \eta \nabla \mathrm{L}(\mathrm{f}(x^{(i)}; \theta), y^{(i)}) \qquad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$

- Note that enough negative examples would eventually make w_2 negative

Mini-batch training

- Stochastic gradient descent chooses a single random example at a time.
- That can result in choppy movements
- More common to compute gradient over batches of training instances.
- **Batch training**: entire dataset
- **Mini-batch training**: *m* examples at a time (512, or 1024)
- Mini-batches can easily be vectorized
- Size of the minibatch based on the computational resources.

➔ Can process all the examples in one mini-batch in parallel and then accumulate the loss

- not possible with individual or batch training.

Regularization

Overfitting

- A model that perfectly matches the training data has a problem.
- It will also **overfit** to the data, modeling noise
  - A random word that perfectly predicts *y* (it happens to only occur in one class) will get a very high weight.
  - Failing to generalize to a test set without this word.
- A good model should be able to **generalize** well to the unseen (test) data
- A model that overfits will have poor generalization

Overfitting

Useful or harmless features

\+

This movie drew me in, and it'll do the same to you.

X1 = "this"
X2 = "movie
X3 = "hated"
X4 = "drew me in"

\-

I can't tell you how much I hated this movie. It sucked.

4-gram features that just "memorize" training set and might cause problems

X5 = "the same to you"
X7 = "tell you how much"

Overfitting

- A 4-gram model on tiny data will just memorize the data

  ➔ 100% accuracy on the training set
- But it will be surprised by the novel 4-grams in the test data

  ➔ Low accuracy on the test set
- Models that are too powerful can **overfit** the data
  - Fitting the details of the training data so exactly that the model doesn't generalize well to the test set
    - How to avoid overfitting?
      - Regularization in logistic regression
      - Dropout in neural networks

Regularization

- A solution for overfitting
- Add a regularization term $R(\theta)$ to the loss function (now written as maximizing logprob rather than minimizing loss)

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)}) - \alpha R(\theta)$$

- Idea: choose an $R(\theta)$ that penalizes large weights
  - Fitting the data well with lots of big weights not as good as fitting the data a little less well, with small weights

L2 Regularization (= Ridge Regression)

- The sum of the squares of the weights
- The name is because this is the (square of the) **L2 norm** $\|\theta\|_2$, = **Euclidean distance** of θ to the origin.

$$R(\boldsymbol{\theta}) = ||\boldsymbol{\theta}||_2^2 = \sum_{j=1}^{n} \boldsymbol{\theta}_j^2$$

- L2 regularized objective function (where α is a hyperparameter):

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)}) \right] - \alpha \sum_{j=1}^{n} \theta_j^2$$

L1 Regularization (= Lasso Regression)

- The sum of the (absolute value of the) weights
- Named after the **L1 norm** $||W||_1$, = sum of the absolute values of the weights, = **Manhattan distance**

$$R(\theta) = ||\theta||_1 = \sum_{i=1}^{n} |\theta_i|$$

- L1 regularized objective function:

$$\hat{\theta} = \underset{\theta}{\text{argmax}} \left[\sum_{1=i}^{m} \log P(y^{(i)}|x^{(i)}) \right] - \alpha \sum_{j=1}^{n} |\theta_j|$$

L1 vs L2 Regularization

- Both are commonly used in language processing.
- L2 regularization: easier to optimize because of its simple derivative (the derivative of θ^2 is just 2θ)
- L1 regularization: more complex (the derivative of $|\theta|$ is non-continuous at zero).
- L2 prefers weight vectors with many small weights
- L1 prefers sparse solutions with some larger weights but many more weights set to zero.

➔ L1 regularization leads to much sparser weight vectors, i.e. far fewer features.

Multinomial Logistic Regression

Multinomial Logistic Regression

- Often we need more than 2 classes
  - Positive/negative/neutral
  - Parts of speech (noun, verb, adjective, adverb, preposition, etc.)
  - Classify emergency SMSs into different actionable classes
- If >2 classes we use **multinomial logistic regression**
  = Softmax regression
  = Multinomial logit (in economics, etc.)
- So "logistic regression" will just mean binary (2 output classes)

Multinomial Logistic Regression

- Recall the expression of the loss for binary LR:

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$$

- Multinomial LR generalizes to K terms.
- y and $\hat{y}$ are represented as vectors.
- True label y is a vector with K elements, each corresponding to a class, with yc = 1 if the correct class is c; all other elements of y being 0.
- Our classifier will produce an estimate vector with K elements $\hat{y}$, each element $\hat{y}_k$ of which represents the estimated probability $p(y_k = 1 | x)$.

Multinomial Logistic Regression

$$
\begin{aligned}
L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) &= -\sum_{k=1}^{K} y_k \log \hat{y}_k \\
&= -\log \hat{y}_c, \quad \text{(where } c \text{ is the correct class)} \\
&= -\log \hat{p}(y_c = 1 | \mathbf{x}) \quad \text{(where } c \text{ is the correct class)} \\
&= -\log \frac{\exp(\mathbf{w_c} \cdot \mathbf{x} + b_c)}{\sum_{j=1}^{K} \exp(\mathbf{w_j} \cdot \mathbf{x} + b_j)} \quad (c \text{ is the correct class})
\end{aligned}
$$

The cross-entropy loss is simply the log of the output probability **corresponding to the correct class** (hence the name negative log likelihood loss).

Multinomial Logistic Regression

- The gradient for a single example turns out to be very similar to the gradient for binary LR:

$$\begin{aligned}
\frac{\partial L_{\text{CE}}}{\partial w_{k,i}} &= -(y_k - \hat{y}_k)x_i \\
&= -(y_k - p(y_k = 1|x))x_i \\
&= -\left(y_k - \frac{\exp(\mathbf{w_k} \cdot \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w_j} \cdot \mathbf{x} + b_j)}\right) x_i
\end{aligned}$$

The softmax function

- The probability of everything must still sum to 1

```
P(positive|doc) + P(negative|doc) +
P(neutral|doc) = 1
```

- Need a generalization of the sigmoid called the **softmax**
  - Takes a vector $z = [z_1, z_2, ..., z_k]$ of k arbitrary values
  - Outputs a probability distribution
    - each value in the range [0,1]
    - all the values summing to 1

The softmax function

Turns a vector $z = [z_1, z_2, \ldots, z_k]$ of k arbitrary values into probabilities

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)} \quad 1 \leq i \leq k$$

The denominator $\sum_{i=1}^{k} e^{z_i}$ is used to normalize all the values into probabilities.

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

The softmax function

- Turns a vector $z = [z_1, z_2, \ldots, z_k]$ of k arbitrary values into probabilities

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

$$[0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

Softmax in multinomial logistic regression

$$p(y=c|x) = \frac{\exp(w_c \cdot x + b_c)}{\sum_{j=1}^{k} \exp(w_j \cdot x + b_j)}$$

- Input is still the dot product between weight vector *w* and input vector *x*
- But now we'll need separate weight vectors for each of the *K* classes.

Features in binary versus multinomial logistic regression

- Binary: positive weight → y=1

  negative weight → y=0

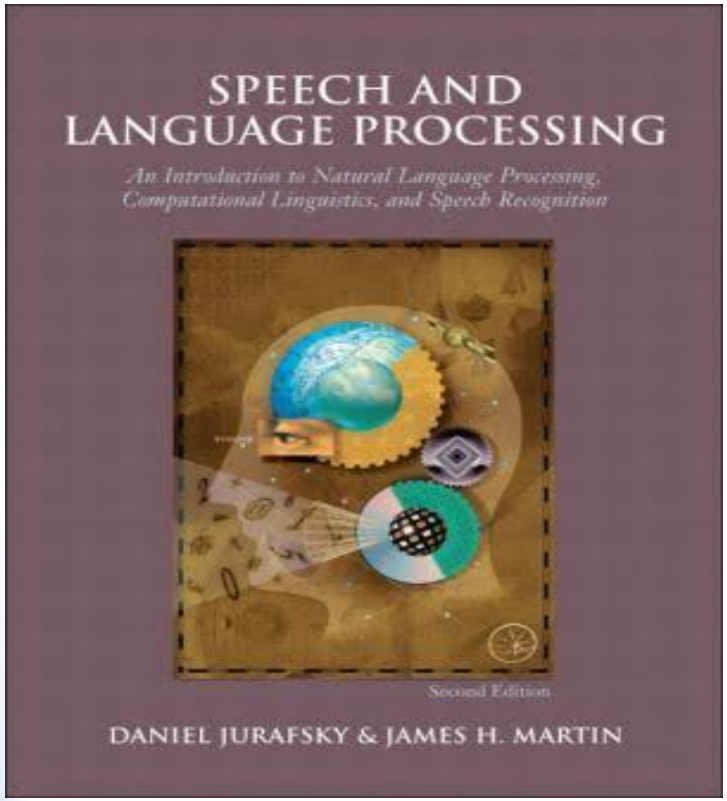
$$x_5 = \begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases} \qquad w_5 = 3.0$$

- Multinominal: separate weights for each class:

Feature	Definition	$w_{5,+}$	$w_{5,-}$	$w_{5,0}$
$f_5(x)$	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	3.5	3.1	−5.3

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.